

Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.

```
#include <stdio.h>

int cost[10][10], parent[10], n;

int find(int x) {
    while (parent[x] != x) x = parent[x];
    return x;
}

int uni(int a, int b) {
    if (find(a) != find(b)) {
        parent[find(b)] = find(a);
        return 1;
    }
    return 0;
}

int main() {
    int i, j, a, b, u, v, min, ne = 1, mincost = 0;

    printf("\n\tImplementation of Kruskal's Algorithm\n");
    printf("\nEnter the number of vertices: ");
    scanf("%d", &n);

    printf("\nEnter the cost adjacency matrix:\n");
    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0) cost[i][j] = 999;
        }

    printf("\nThe edges of Minimum Cost Spanning Tree are:\n");

    while (ne < n) {
        min = 999;

        for (i = 1; i <= n; i++)
            for (j = 1; j <= n; j++)
```

```
        if (cost[i][j] < min) {
            min = cost[i][j];
            a = u = i;
            b = v = j;
        }

    u = find(u);
    v = find(v);

    if (uni(u, v)) {
        printf("%d edge (%d,%d) = %d\n", ne++, a, b, min);
        mincost += min;
    }

    cost[a][b] = cost[b][a] = 999;
}

printf("\nMinimum cost = %d\n", mincost);
return 0;
}
```

Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm

```
#include <stdio.h>

int cost[10][10], n, mincost = 0;

int prims() {
    int near[10], i, j, k, u, min;

    for (i = 2; i <= n; i++) near[i] = 1;
    near[1] = 0;

    for (i = 1; i < n; i++) {
        min = 999;

        for (j = 1; j <= n; j++)
            if (near[j] && cost[j][near[i]] < min) {
                min = cost[j][near[i]];
                u = j;
            }

        mincost += min;
        printf("The Minimum edge is %d -> %d \t cost = %d\n",
            near[u], u, min);

        near[u] = 0;

        for (k = 1; k <= n; k++)
            if (near[k] && cost[k][near[u]] > cost[k][u])
                near[k] = u;
    }

    return mincost;
}

int main() {
    int i, j;

    printf("***** PRIMS ALGORITHM *****\n");
```

```
printf("Enter the number of nodes:\n");
scanf("%d", &n);

printf("Enter the cost matrix:\n");
for (i = 1; i <= n; i++)
    for (j = 1; j <= n; j++)
        scanf("%d", &cost[i][j]);

printf("Minimum Spanning Tree Edges and costs are:\n");
mincost = prims();

printf("The minimum spanning tree cost is %d\n", mincost);

return 0;
}
```

Design and implement C/C++ Program to find shortest paths from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm

```
#include <stdio.h>

void dij(int s, int cost[20][20], int d[20], int n) {
    int visited[20] = {0}, i, j, u, min, w;

    for (i = 1; i <= n; i++)
        d[i] = cost[s][i];

    visited[s] = 1;
    d[s] = 0;

    for (j = 2; j <= n; j++) {
        min = 999;

        for (i = 1; i <= n; i++)
            if (!visited[i] && d[i] < min)
                min = d[u = i];

        visited[u] = 1;

        for (w = 1; w <= n; w++)
            if (!visited[w] && d[w] > d[u] + cost[u][w])
                d[w] = d[u] + cost[u][w];
    }
}

int main() {
    int i, j, n, source;
    int cost[20][20], d[20];

    printf("Enter no. of vertices: ");
    scanf("%d", &n);

    printf("Enter the cost adjacency matrix:\n");
    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);
```

```
        if (cost[i][j] == 0 && i != j)
            cost[i][j] = 999;
    }

    printf("\nEnter the source node: ");
    scanf("%d", &source);

    dij(source, cost, d, n);

    for (i = 1; i <= n; i++)
        if (i != source)
            printf("\nShortest path from %d to %d is %d",
                source, i, d[i]);

    return 0;
}
```

5. Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>

int a[10][10], indegree[20], n;

void topological() {
    int stack[20], top = -1;
    int t[20], k = 0, i, j, u;

    for (j = 1; j <= n; j++)
        for (i = 1; i <= n; i++)
            indegree[j] += a[i][j];

    for (i = 1; i <= n; i++)
        if (indegree[i] == 0)
            stack[++top] = i;

    while (top != -1) {
        u = stack[top--];
        t[k++] = u;

        for (i = 1; i <= n; i++)
            if (a[u][i] && --indegree[i] == 0)
                stack[++top] = i;
    }

    printf("\nTopological Ordering is:\n");
    for (i = 0; i < k; i++)
        printf("%d ", t[i]);
}

int main() {
    int i, j;

    printf("Enter the number of nodes: ");
    scanf("%d", &n);

    printf("\nEnter the adjacency matrix:\n");
```

```
for (i = 1; i <= n; i++)  
    for (j = 1; j <= n; j++)  
        scanf("%d", &a[i][j]);  
  
topological();  
return 0;  
}
```

Design and implement C/C++ Program to solve 0/1 Knapsack problem using Dynamic Programming method.

```
#include <stdio.h>

#define MAX 50

int p[MAX], w[MAX], n;

int knapsack(int i, int m) {
    if (i == n)
        return (w[n] <= m) ? p[n] : 0;

    if (w[i] > m)
        return knapsack(i + 1, m);

    int a = knapsack(i + 1, m);
    int b = knapsack(i + 1, m - w[i]) + p[i];

    return (a > b) ? a : b;
}

int main() {
    int m, i;

    printf("Enter no. of objects: ");
    scanf("%d", &n);

    printf("\nEnter the weights:\n");
    for (i = 1; i <= n; i++)
        scanf("%d", &w[i]);

    printf("\nEnter the profits:\n");
    for (i = 1; i <= n; i++)
        scanf("%d", &p[i]);

    printf("\nEnter the knapsack capacity: ");
    scanf("%d", &m);

    printf("\nThe optimal solution is: %d\n", knapsack(1, m));
```

```
return 0;
```

```
}
```

Design and implement C/C++ Program to solve discrete Knapsack and continuous Knapsack problems using greedy approximation method.

```
#include <stdio.h>

int main() {
    float w[50], p[50], r[50];
    float cap, value = 0, temp;
    int n, i, j;

    printf("Enter the number of items: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter Weight and Profit for item[%d]:\n", i);
        scanf("%f %f", &w[i], &p[i]);
        r[i] = p[i] / w[i];
    }

    printf("Enter the capacity of knapsack:\n");
    scanf("%f", &cap);

    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (r[i] < r[j]) {
                temp = r[i]; r[i] = r[j]; r[j] = temp;
                temp = w[i]; w[i] = w[j]; w[j] = temp;
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }

    printf("\nKnapsack problem using Greedy Algorithm:\n");

    for (i = 0; i < n && cap > 0; i++) {
        if (w[i] <= cap) {
            value += p[i];
            cap -= w[i];
        } else {
            value += r[i] * cap;
            break;
        }
    }
}
```

```
}  
  
printf("\nThe maximum value is: %f\n", value);  
return 0;  
}
```

Design and implement C/C++ Program to find a subset of a given set $S=\{s_1,s_2,\dots,s_n\}$ of n positive integers whose sum is equal to a given positive integer d .

```
#include <stdio.h>

int w[10], x[10], d, count = 0, n;

void subset(int cs, int k, int r) {
    int i;

    x[k] = 1;

    if (cs + w[k] == d) {
        printf("\nSubset %d:\n", ++count);
        for (i = 0; i <= k; i++)
            if (x[i]) printf("%d\t", w[i]);
    }
    else if (k + 1 < n && cs + w[k] + w[k + 1] <= d)
        subset(cs + w[k], k + 1, r - w[k]);

    if (k + 1 < n && cs + r - w[k] >= d && cs + w[k] <= d) {
        x[k] = 0;
        subset(cs, k + 1, r - w[k]);
    }
}

int main() {
    int i, sum = 0;

    printf("Enter the no. of elements: ");
    scanf("%d", &n);

    printf("\nEnter the elements in ascending order:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &w[i]);
        sum += w[i];
    }

    printf("\nEnter the sum: ");
    scanf("%d", &d);
```

```
if (sum < d) {  
    printf("No solution\n");  
    return 0;  
}  
  
subset(0, 0, sum);  
  
if (!count)  
    printf("No solution\n");  
  
return 0;  
}
```

Design and implement C/C++ Program to sort a given set of n integer elements using Selection Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void selectionSort(int a[], int n) {
    int i, j, min, t;

    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++)
            if (a[j] < a[min])
                min = j;

        t = a[i];
        a[i] = a[min];
        a[min] = t;
    }
}

int main() {
    int n = 6000, i;
    int *a = malloc(n * sizeof(int));

    srand(time(NULL));

    printf("Random numbers for n = %d:\n", n);
    for (i = 0; i < n; i++) {
        a[i] = rand() % 1000;
        printf("%d ", a[i]);
    }

    clock_t start = clock();

    selectionSort(a, n);
```

```
clock_t end = clock();

printf("\nTime taken to sort for n = %d: %lf seconds\n",
      n, (double)(end - start) / CLOCKS_PER_SEC);

printf("\nSorted numbers for n = %d:\n", n);
for (i = 0; i < n; i++)
    printf("%d ", a[i]);

free(a);
return 0;
}
```

Design and implement C/C++ Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n > 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int a[], int low, int high) {
    int pivot = a[high], i = low - 1, j;

    for (j = low; j < high; j++)
        if (a[j] < pivot)
            swap(&a[++i], &a[j]);

    swap(&a[i + 1], &a[high]);
    return i + 1;
}

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

int main() {
    int n = 6000, i;
    int *a = malloc(n * sizeof(int));

    srand(time(NULL));

    printf("Random numbers for n = %d:\n", n);
```

```
for (i = 0; i < n; i++) {  
    a[i] = rand() % 1000;  
    printf("%d ", a[i]);  
}  
  
clock_t start = clock();  
  
quickSort(a, 0, n - 1);  
  
clock_t end = clock();  
  
printf("\nTime taken to sort for n = %d: %lf seconds\n",  
    n, (double)(end - start) / CLOCKS_PER_SEC);  
  
printf("\nSorted numbers for n = %d:\n", n);  
for (i = 0; i < n; i++)  
    printf("%d ", a[i]);  
  
free(a);  
return 0;  
}
```